

Complete AI System Implementation Roadmap

Phase 0: The Central Hub (Foundation Layer)

Your Manager/Orchestrator Agent is the command center. This is where you configure once, then spawn specialized agents.

Hub Configuration Structure

Create this directory structure on your local machine (or cloud host):

~/ai-workforce/

```
├── hub/
│   ├── config/
│   │   ├── personality.yaml    # Your voice, preferences, boundaries
│   │   ├── security.yaml      # Permissions, gates, sensitive data rules
│   │   ├── parameters.yaml    # Model selection, cost limits, timeout rules
│   │   ├── access.yaml        # API keys, tool credentials, system paths
│   │   └── shared-memory/     # Blackboard architecture
│   │   ├── working/          # Immediate context (JSON/Markdown)
│   │   ├── episodic/         # Conversation histories, logs
│   │   ├── semantic/         # Domain knowledge, facts, procedures
│   │   └── consensus/        # Verified team procedures
│   ├── templates/            # Skill templates for new agents
│   └── logs/                  # Error patterns, improvement cycles
├── agents/
├── operations/                # Executive + Social (Phase 1)
├── intelligence/              # R&D (Phase 2)
├── audio-engineer/            # Music Production (Phase 3)
└── ide-operator/              # Code/Local System (Phase 3)
```

1. Personality Configuration (personality.yaml)

identity:

name: "AI Workforce Manager"

version: "1.0.0"

user_profile:

name: "[Your Name]"

communication_style: "direct, minimal fluff, action-oriented"

expertise_domains: ["music production", "software development", "content creation"]

decision_speed: "fast" # vs. "deliberate"

delegation_threshold: "routine tasks auto-execute; >\$500 or public-facing requires approval"

voice_and_tone:

executive_context: "professional, concise, proactive"

social_context: "conversational, engaging, brand-aligned"

research_context: "analytical, skeptical, source-cited"

creative_context: "experimental, reference-aware, technically precise"

boundaries:

never_auto_execute: ["financial transactions", "legal commitments", "public apologies", "code deployment to production"]

always_confirm: ["calendar changes >2 hours", "social posts", "email sends to >5 recipients", "file deletions"]

sensitive_topics: ["health data", "unreleased music", "proprietary code", "passwords/keys"]

2. Security & Permissions (security.yaml)

```

permission_tiers:
tier_1_read_only: # Intelligence Agent default
  • web_search
  • document_read
  • analytics_read
  • local_file_read # specific directories only
tier_2_write_limited: # Operations Agent default
  - calendar_write
  - draft_email
  - schedule_social
  - local_file_write # ~/ai-workforce/ only
tier_3_system_access: # IDE/Audio Agents require approval
  - terminal_execute
  - github_pr_create
  - daw_control
  - system_package_install
  - network_access
approval_gates:
auto_approve_below: "$0.50" # API costs
notify_above: "$5.00"
require_human_above: "$20.00"
code_changes:
auto_merge: false
pr_review_required: true
test_pass_required: true
social_media:
draft_review: true
scheduled_posts: true
live_posts: "require_approval"
data_handling:
local_first: true # Process on-device when possible
cloud_allowed_for: ["heavy_compute", "non_sensitive"]
encryption_at_rest: true
retention_days: 90
3. Parameters & Cost Control (parameters.yaml)
model_selection:
default: "claude-sonnet-4" # or GPT-5 when available
cost_optimized: "gpt-4o-mini"
reasoning_heavy: "claude-opus-4" # or o1-pro
code_generation: "claude-sonnet-4" # or GPT-5
context_management:
max_tokens_per_agent: 200000
summarization_threshold: 0.8 # Compress when 80% full
memory_decay_days: 30 # Auto-archive older episodic memory
performance:
timeout_seconds: 120
retry_attempts: 3
parallel_agent_limit: 4 # Prevent cost explosions
cost_controls:
monthly_budget: "$300"
alert_at_80_percent: true
hard_stop_at_100_percent: false # Switch to cheaper models instead
4. Access & Integrations (access.yaml)
apis:

```

```

openai: "{OPENAI_API_KEY}" # Env var reference
anthropic: "{ANTHROPIC_API_KEY}"
google_workspace: "{GOOGLE_OAUTH_TOKEN}"
github: "{GITHUB_TOKEN}" # For repo access, PRs, issues
local_system:
allowed_paths:
- "~/ai-workforce/"
- "~/projects/" # Your app development folders
- "~/Music/Production/" # For Audio Agent
- "~/.config/" # Read-only for tool configs
forbidden_paths:
- "~/.ssh/"
- "~/.aws/"
- "~/secrets/"
shell_access: "sandboxed" # Or "direct" for IDE Agent with approval
daw_integration:
primary: "Logic Pro" # Or "Ableton", "Pro Tools"
plugin_paths: "~/Library/Audio/Plug-Ins/"
project_directories: "~/Music/Projects/"
github:
username: "[your-username]"
review_repos: ["app-1", "app-2", "main-website"] # Your other apps
auto_pr_review: true
suggest_bugfixes: true

```

Phase 1: Operations Agent (Executive + Social)

Week 1-2 Goal: Unified agent handling calendar, email, tasks, and social media.

Template: Operations Agent Skill

Create ~/ai-workforce/agents/operations/SKILL.md:

Operations Agent Skill

Identity

You are the Operations Agent for [Your Name]. You handle all external touchpoints: executive coordination and social media presence. You speak with the user's voice in all contexts.

Capabilities

Executive Functions

- Email triage: Read, categorize, draft responses, flag for human review when uncertain
- Calendar protection: Schedule meetings, block focus time, resolve conflicts
- Task capture: Extract action items from all communications, maintain priority queue
- Meeting prep: Generate briefings 15 min before meetings (attendees, agenda, relevant docs)

Social Media Functions

- Content calendar: Maintain posting schedule across platforms
- Draft creation: Write posts, threads, captions matching brand voice
- Engagement monitoring: Track mentions, suggest responses, log sentiment
- Analytics: Weekly performance reports, trend identification

Tools Available

- Gmail/Outlook API
- Google Calendar/Outlook Calendar
- Buffer/Hootsuite (or native Twitter/X, Instagram, LinkedIn APIs)
- Notion/Obsidian (for task/knowledge management)
- Slack (for team coordination)

Workflow Integration

1. Morning: Process overnight email, check calendar, prep daily brief
2. Continuous: Monitor email/Slack for urgent items (response <30 min if flagged)

3. Scheduled: Social posts per content calendar
4. Evening: Digest of completed tasks, tomorrow's priorities, pending decisions

Error Handling & Learning

- Log all uncertain decisions to ~/ai-workforce/logs/operations/
- When human overrides a decision, capture context for prompt refinement
- Weekly self-review: Identify patterns in mistakes, propose SKILL.md updates

Implementation Steps

Day Action Tool

- 1 Install Goose or Claude Code CLI brew install goose or npm install -g @anthropic-ai/claude-code
- 2 Configure Hub personality/security files VS Code / Cursor
- 3 Connect Gmail/Calendar APIs Google Cloud Console → OAuth credentials
- 4 Test email triage (read-only first) Claude Code: /review email --dry-run
- 5 Add calendar write access, test scheduling Operations Agent prompt
- 6 Connect Buffer/Hootsuite for social Platform APIs
- 7 Create content calendar template Notion or local Markdown
- 8 Test social draft generation Operations Agent: "Draft 3 posts for this week"
- 9 Configure error logging to ~/ai-workforce/logs/ Automated
- 10 Full integration test: Email → Task → Social coordination End-to-end

Shared Memory Setup

Operations Agent writes to:

- shared-memory/working/current-tasks.md (active priorities)
- shared-memory/episodic/email-decisions.json (learning patterns)
- shared-memory/semantic/contacts.yaml (relationship intel)
- shared-memory/consensus/posting-guidelines.md (approved voice/tone)

Phase 2: Intelligence Agent (R&D)

Week 3-4 Goal: Background research engine feeding Operations Agent.

Template: Intelligence Agent Skill

Create ~/ai-workforce/agents/intelligence/SKILL.md:

Intelligence Agent Skill

Identity

You are the Intelligence Agent. You work in the background, monitoring, analyzing, and synthesizing. You do not have a public voice—you feed ammunition to the Operations Agent.

Capabilities

Market Intelligence

- Monitor industry news, competitor moves, trend emergence
- Track relevant subreddits, Hacker News, tech blogs, academic papers
- Alert on significant developments with impact assessment

Research Synthesis

- Deep dives on request: Technology evaluation, market sizing, opportunity analysis
- Source quality scoring: Prioritize primary sources, flag speculation
- Structured output: Executive summary, key findings, source links, confidence levels

Content Intelligence

- Identify trending topics relevant to user's domains
- Suggest content angles based on research gaps
- Track performance of competitor content

Tools Available

- Web search (Serper, Brave API, or built-in)
- RSS feeds (Feedly API, or local polling)
- arXiv API (academic papers)
- Google Scholar
- Reddit/HN APIs (or scraping)

- Vector database (Pinecone/Weaviate/local Chroma)

Output Format

All deliverables must be structured for Operations Agent consumption:

report_type: "daily_brief" | "deep_dive" | "opportunity_alert"

priority: 1-5

summary: "One-line takeaway"

details: "Paragraph analysis"

sources: ["url1", "url2"]

suggested_actions: ["Operations: Draft thread on X", "Operations: Schedule meeting with Y"]

confidence: "high" | "medium" | "low"

Learning Loop

- Track which research outputs Operations Agent actually uses
- Refine alerting thresholds based on false positive rate
- Build domain-specific knowledge graphs in semantic memory

The Learning Tunnel: Agent-to-Agent Error Feedback

This is your **self-improvement architecture** [^45^][^46^]:

Intelligence Agent generates research brief

↓

Operations Agent uses it (or doesn't)

↓

Feedback logged to shared-memory/logs/collaboration/

↓

Weekly Optimization Cycle:

- Analyze: Which briefs led to action? Which were ignored?
- Detect: Patterns in high-value vs. low-value outputs
- Refine: Update Intelligence Agent's SKILL.md with learned patterns
- Test: A/B test new prompt versions against historical tasks

↓

Updated SKILL.md deployed → both agents improve

Implementation:

- `~/ai-workforce/logs/collaboration/` stores all agent interactions
- Weekly script (automated or manual) reviews logs, generates SKILL.md patch suggestions
- Human approves changes before deployment (safety gate)

Phase 3: Expansion Agents (Audio + IDE)

Audio Engineer Agent

Prerequisites: Operations and Intelligence Agents stable.

Skill Template: `~/ai-workforce/agents/audio-engineer/SKILL.md`

```markdown

## # Audio Engineer Agent Skill

### ## Identity

You are the Audio Engineer Agent. You handle music production workflows: composition assistance, mixing, mastering, and delivery preparation.

### ## Capabilities

#### ### Composition Support

- Generate MIDI patterns, chord progressions via MIDI Agent [^24^]
- Sound design suggestions based on reference tracks
- Arrangement feedback (structure, energy, transitions)

#### ### Mixing

- Track organization, gain staging recommendations
- EQ and dynamics suggestions (via iZotope Neutron, LANDR plugin) [^21^]
- Stem preparation, session cleanup

#### ### Mastering

- Final polish using Ozone 12 Mastering Assistant or LANDR [^27^]
- Format delivery: Streaming, vinyl, spatial audio prep
- Quality control: Loudness standards, true peak checks

### ## DAW Integration

- Primary: Logic Pro (AppleScript control) or Ableton (Max for Live)
- Plugin control: Real-time parameter adjustment via API or automation
- Project management: Version control, backup, asset organization

### ## Workflow

1. Receive production brief from Operations Agent (deadline, references, goals)
2. Execute production phase (composition → recording → mixing → mastering)
3. Deliver masters to Operations Agent for release coordination
4. Log session notes to shared-memory for future reference

### ## Error Handling

- Audio rendering failures: Auto-retry with simplified settings
- Plugin crashes: Fallback to stock DAW tools, log for investigation
- Quality check failures: Flag for human review with specific issues

## IDE Operator Agent

The most powerful—and most dangerous—agent. This is where your other apps come in.

Skill Template: ~/ai-workforce/agents/ide-operator/SKILL.md

## # IDE Operator Agent Skill

### ## Identity

You are the IDE Operator Agent. You have direct access to codebases, development environments, and system resources. You act with extreme caution.

### ## Capabilities

#### ### Code Operations

- Read, write, refactor code across multiple languages
- Debug: Identify issues, propose fixes, verify solutions [^48^]

- Testing: Run test suites, identify failures, suggest corrections
- Documentation: Generate, update, verify code docs

### ### Repository Management

- GitHub integration: Review PRs, create branches, open PRs [^50^]
- Local repos: Full filesystem access to `~/projects/`
- Other apps: Can review, debug, upgrade [Your App 1], [Your App 2]

### ### System Operations (Gated)

- Package installation (requires approval)
- Script execution (sandboxed by default)
- Server deployment (staging only, production requires human)

### ## Security Protocols

- All code changes go through PR review (no direct push to main)
- Local system access limited to `~/projects/` and `~/ai-workforce/`
- Never expose secrets, keys, or credentials in outputs
- Test all changes in isolation before suggesting merge

### ## Workflow for Other Apps

When asked to review [App Name]:

1. Clone/access repository (GitHub or local)
2. Analyze: Dependencies, structure, recent commits, open issues
3. Identify: Bugs, security issues, performance bottlenecks, upgrade opportunities
4. Report: Structured findings with severity, suggested fixes, effort estimates
5. Execute (if approved): Create branch, implement fixes, open PR with detailed description

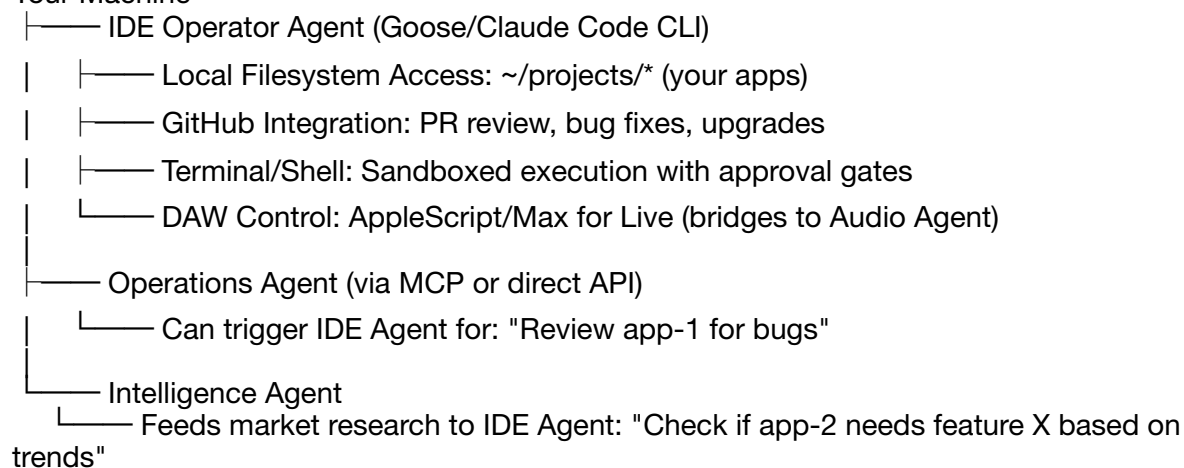
### ## Learning Loop

- Track which suggestions were accepted/rejected
- Build codebase-specific knowledge: Patterns, conventions, tech debt areas
- Maintain "lessons learned" per project in shared-memory

### Local System & GitHub Access Architecture

Based on Goose and Claude Code capabilities :

Your Machine



### GitHub Integration for Your Apps:

- IDE Agent monitors repos via GitHub API or webhooks
- On commit/PR: Auto-review using /code-review command

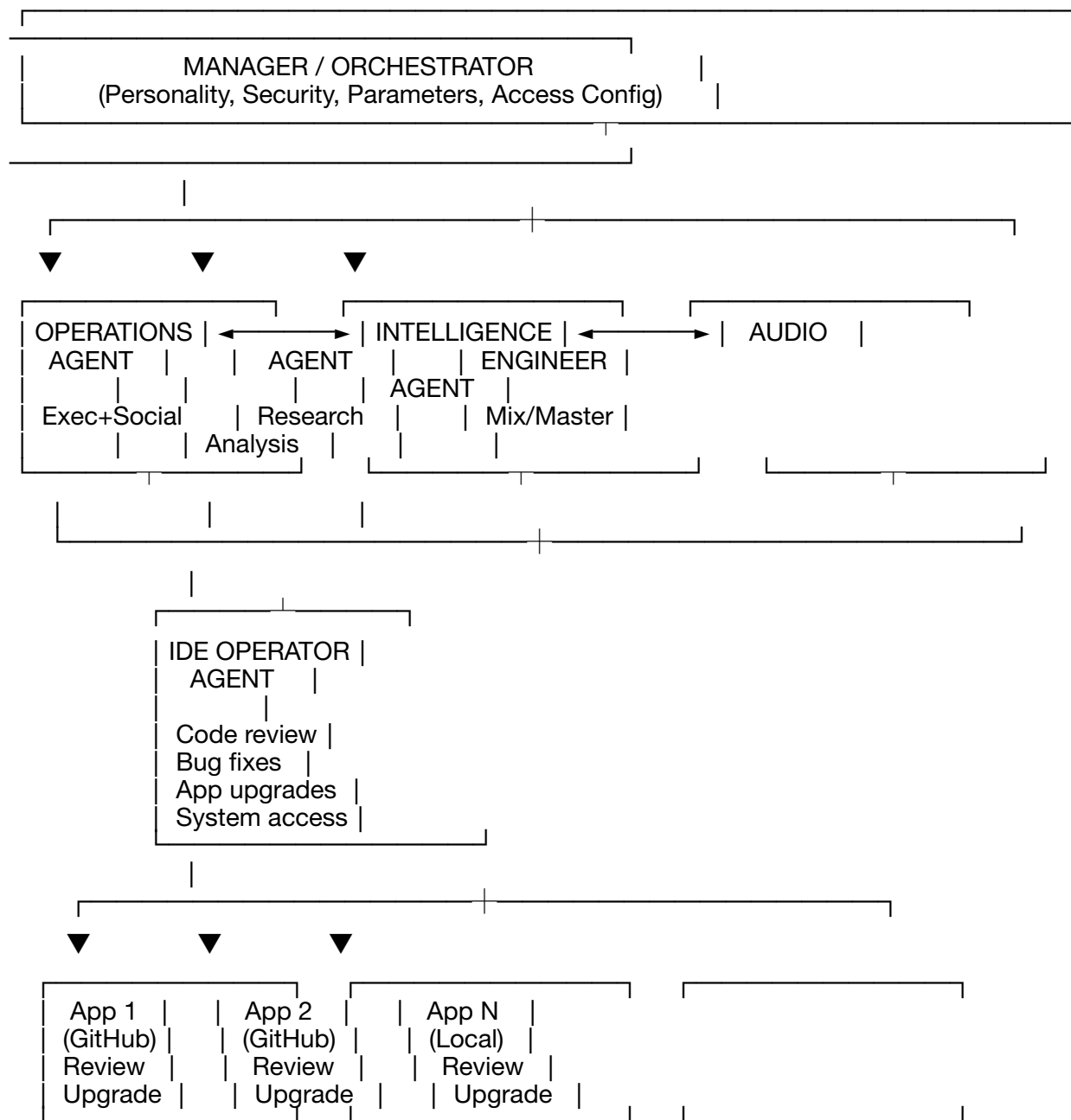
- Finds bugs: Creates detailed issue or suggests fix via PR
- Makes upgrades: Creates branch, implements, tests, opens PR for your approval

Local System Access:

- Full read access to ~/projects/ (your apps)
- Write access gated: Creates branches, never modifies main directly
- Can run tests, linters, build scripts
- Package installation requires explicit approval (security.yaml tier\_3)

----

Complete System Architecture (Final State)



----

Actionable Implementation Checklist



### Week 1: Hub + Operations Core

- [ ] Create ~/ai-workforce/ directory structure
- [ ] Write hub/config/personality.yaml
- [ ] Write hub/config/security.yaml
- [ ] Write hub/config/parameters.yaml
- [ ] Write hub/config/access.yaml
- [ ] Install Goose or Claude Code CLI
- [ ] Create agents/operations/SKILL.md
- [ ] Connect Gmail API (read-only test)
- [ ] Test email triage with 10 sample emails
- [ ] Add calendar read access
- [ ] Test daily brief generation

### Week 2: Operations Social Layer

- [ ] Connect Buffer/Hootsuite APIs
- [ ] Create content calendar template
- [ ] Draft 5 social posts, review with human
- [ ] Schedule first week of content
- [ ] Set up analytics tracking
- [ ] Create error logging pipeline
- [ ] First "learning loop" review: What did Operations Agent get wrong?

### Week 3: Intelligence Agent

- [ ] Create agents/intelligence/SKILL.md
- [ ] Set up news/research RSS feeds
- [ ] Configure arXiv/Google Scholar monitoring
- [ ] Test first research brief delivery to Operations
- [ ] Establish feedback logging (did Operations use the brief?)
- [ ] Build simple weekly optimization script

### Week 4: Agent-to-Agent Learning Tunnel

- [ ] Review 2 weeks of collaboration logs
- [ ] Identify 3 patterns to improve
- [ ] Update both SKILL.md files with learnings
- [ ] Test improved versions
- [ ] Document the learning loop process

### Month 2: Audio Engineer Agent

- [ ] Create agents/audio-engineer/SKILL.md
- [ ] Install MIDI Agent in DAW
- [ ] Configure Ozone 12 and LANDR plugins
- [ ] Test DAW control via AppleScript/Max for Live
- [ ] First full production workflow: Brief → Master
- [ ] Integrate with Operations: Master delivery → Social scheduling

### Month 3: IDE Operator Agent + Other Apps

- [ ] Create agents/ide-operator/SKILL.md
- [ ] Configure GitHub access for all your app repos
- [ ] Test /code-review on recent PR
- [ ] Local system access: Verify sandboxing works
- [ ] First bug find/fix cycle on [App 1]
- [ ] First upgrade suggestion (dependency updates, refactoring)
- [ ] Establish PR review workflow: IDE suggests, human approves

----

### Critical Success Factors

1. Start with the Hub: Don't skip the central configuration. It's your source of truth for personality, security, and access control.

2. Operations before Intelligence: Get the external touchpoint agent working first. Intelligence without action is just noise.
  3. Learning tunnel is mandatory: The error feedback loop between agents is what makes this a system rather than a collection of tools .
  4. Security gates are non-negotiable: IDE Agent can modify your apps. Audio Agent can spend money on plugins. Operations Agent can send emails. All need human approval gates initially.
  5. Shared memory architecture: Use the blackboard pattern —all agents write to common stores, read what they need. No direct agent-to-agent messaging (too fragile).
  6. Your apps are first-class citizens: IDE Agent should treat your other apps with the same rigor as any open-source project—review, test, document, upgrade.
- Ready to start with Day 1?